

Modül 11: Jenerikler

Jenerikler bütün bir başarısızlık sınıfını çalışma zamanından derleme zamanına taşıdı. Java 5'ten önce her koleksiyon `Object` tutuyordu ve her okuma, doğru yapmak zorunda olduğunuz bir dönüşüm gerektiriyordu.

Özelliğin kullanımı kolay, yanlış okunması da kolaydır, çünkü 2004'te verilen tek bir tasarım kararı karşılaşılabileceğiniz neredeyse her kısıtı açıklar. O karar tip silmedir ve bu modül onunla başlıyor.

Neler Öğreneceksiniz

- Jenerik sınıflar ve metotlar, ve sınırların onları neden işe yarar kıldığı
- Tip silme, neyi yasakladığı ve yine de neden seçildiği
- Ham tipler, yığın kirlenmesi ve ortaya çıkan çökmenin gerçekte nereye düştüğü
- PECS üzerinden joker karakterler
- Jeneriklerin neden değişmez, dizilerin neden değişken olduğu ve hangi seçimin hata olduğu

Başka Bir Dilden Geliyorsanız

	Python	Java
Tip parametreleri	ipucu, zorlanmaz	derleme zamanında zorlanır
Çalışma zamanında	<code>__orig_class__</code> ile erişilebilir	silinmiş, hiçbir şey kalmaz
Değişkenlik	TypeVar üzerinde işaretlenir	kullanım yerinde joker karakterle
İlkel tipler	ayrım yok	<code>List<int></code> yasadışı

Sizi şaşırtacak olan satır tip silme. Java'da tip argümanı çalışma zamanında gerçekten var olmaz, ve şaşırtıcı sayıda kısıt bu tek olgudan çıkar.

Ders

Temeller

```
class Box<T> {  
    private final T value;  
    T get() { return value; }  
}  
  
static <T> T first(List<T> items) { return items.get(0); }
```

Jenerik bir metotta açılı parantezler dönüş tipinden **önce** gelir ve tip argümanlardan çıkarılır.

Sınırlı tip parametreleri, jenerik bir metodun değerleri oradan oraya taşımaktan fazlasını yapmasını sağlayan şeydir:

```
static <T extends Comparable<T>> T max(List<T> items)
```

Sınır olmadan `T` bir `Object` gibi ele alınır ve `compareTo` diye bir şey yoktur. Birden fazla sınır `&` kullanır, sınıf önce gelir: `<T extends Number & Comparable<T>>`.

Geleneksel adlar `T` tip, `E` eleman, `K` anahtar, `V` değer, `R` sonuç. Bunlar yalnızca gelenektir.

Tip silme

`Eraseure.java` dosyasından:

```
List<String> class: java.util.ArrayList
List<Integer> class: java.util.ArrayList
same class: true
```

Tip argümanı hiçbir iz bırakmadı. **Neden böyle çalışıyor:** jenerikler JVM'den on beş yıl sonra geldi. Tip silme, jenerik ve jenerik öncesi kodun birbirini çağırabilmesi ve değişmemiş bir JVM üzerinde çalışabilmesi demektir, böylece mevcut kütüphanelerin yeniden yazılması gerekmedi. Bilinçli bir takastı: o zaman uyumluluk, karşılığında sonsuza dek bu kısıtlar.

Tip silmenin yasakladıkları:

Yasak	Sebebi
<code>x instanceof List<String></code>	çalışma zamanında öyle bir tip yok
<code>new T[]</code> , <code>new T()</code>	tahsis edilecek bir şey yok
<code>show(List<String>)</code> ve <code>show(List<Integer>)</code>	aynı silme, çakışmalar
<code>List<int></code>	ilkel tipler nesne değil
<code>Box<T></code> içinde <code>static T shared;</code>	sınıf başına tek statik alan, tip argümanı başına değil

Çalışma zamanında tipe gerçekten ihtiyacınız olduğunda standart çözüm bir `Class<T>` geçirmektir, kütüphanedeki bu kadar çok metodun birini almasının sebebi budur.

Ham tipler ve yığın kirlenmesi

`HeapPollution.java` bir uyarıyla derlenir ve sonra başarısız olur:

```
List<String> words = new ArrayList<>();
List raw = words; // raw type, legal for backward compatibility
raw.add(42); // unchecked warning, allowed
```

```
the list now contains: [legitimate, 42]
reading element 0 is fine: legitimate
reading element 1 will not be:
Exception in thread "main" java.lang.ClassCastException:
    class java.lang.Integer cannot be cast to class java.lang.String
```

Çökmenin nereye düştüğüne bakın. Kuralı bozan satır `raw.add(42)` idi. Başarısızlık `get` çağrısında, çünkü derleyici bildirilen tipin söz verdiği dönüşümü oraya koydu. Gerçek kodda bu iki satır genellikle farklı kişilerce yazılmış farklı sınıflardadır, ve o mesafe yığın kirlenmesini hata ayıklaması zor kılan şeydir.

Yeni kodda asla ham tip kullanmayın. Unchecked uyarıları, derleyicinin artık hiçbir şeyi garanti edemez hâle geldiği anlamına gelir. Birini bastırmak zorunda kaldığınızda `@SuppressWarnings` anotasyonunu mümkün olan en dar kapsama koyun ve neden güvenli olduğunu söyleyin.

Joker karakterler: PECS

Producer Extends, Consumer Super.

Jenerikler **değişmezdir**: `Integer` bir `Number` olmasına rağmen `List<Integer>` bir `List<Number>` değildir. Joker karakterler olmadan eleman tipi başına bir aşırı yükleme gerekirdi.

```
// PRODUCER: reads from the list, so any subtype of Number will do
static double sum(List<? extends Number> numbers)

// CONSUMER: writes into the list, so any supertype of Integer will do
static void addAll(List<? super Integer> destination, List<Integer> source)
```

Her yön size değerini kaybettirir:

- `? extends` **salt okunurdur**. Liste bir `List<Double>` olabilir, dolayısıyla bir `Integer` eklemek onu bozardı. Yalnızca `null` eklenebilir.
- `? super` **Object olarak okunur**. Liste bir `List<Object>` olabilir, dolayısıyla derleyicinin söz verebileceği tek şey budur.

`List<?>` sınırsız biçimdir: `Object` okuyun, boyuta bakın, hiçbir şey eklemeyin. Ham `List` tipinden önemli olan biçimde ayrılır, çünkü `List<?>` tip güvenlidir ve zorlanır, ham tip ise denetimi tamamen kapatır.

Değişmezlik doğru karardı

Diziler öteki cevabı seçti. `Wildcards.java` dosyasından:

```
Object[] array = new String[2]; // compiles: arrays are covariant
array[0] = 42;                 // throws at runtime
```

```
ArrayStoreException: java.lang.Integer
```

O karar yüzünden her dizi yazma işlemi bir çalışma zamanı tip denetimi taşır. Jenerikler eşdeğerini derleme zamanında reddeder, dolayısıyla hiçbir denetim gerekmez ve hata üretime ulaşamaz.

Çalıştırın

```
java modules/11-generics/examples/Generics.java
java modules/11-generics/examples/Erasure.java
java modules/11-generics/examples/Wildcards.java

# Bu ikisi bilerek başarısız olur, biri çalışma zamanında biri derleme zamanında.
java modules/11-generics/examples/HeapPollution.java
java modules/11-generics/examples/CannotCreateGenericArray.java

./scripts/verify-examples.sh modules/11-generics
```

Sık Yapılan Hatalar

Bir uyarıyı susturmak için ham tip kullanmak. Özelliğin var olma sebebi olan denetimi kapattınız.

Tüm sınıfa ya da metoda `@SuppressWarnings` koymak. O kapsamdaki gelecekteki her hatayı da gizler. Yalnızca ihtiyaç duyan tek bildirime koyun.

`new T[size]` denemek. Bir `List` kullanın, ya da `Object[]` tahsis edip dönüştürün ve diziyi sınıfa özel tutun.

Eklemeniz gereken bir parametre için `List<? extends Number>` yazmak. Ekleyemezsiniz. Hem okuyup hem yazıyorsanız düz bir `List<T>` kullanın.

Dönüş tipinde joker karakter. Her çağıranı joker karakterle uğraşmaya zorlar. Somut tipi döndürün.

Tip parametresi yeterken `Object` kullanmak. O zaman her çağıranın bir dönüşüme ihtiyacı olur, ki bu tam olarak 2004 öncesi durumdur.

Özet Çıkarımlar

- **Jenerikler derleme zamanında denetlenir ve çalışma zamanında silinir.** Neredeyse her kısıt bundan çıkar.
- **Sınırlar, jenerik kodun değer taşımanın ötesinde bir şey yapmasını sağlar.**
- **Ham tipler özelliğin tamamını bozar,** ve ortaya çıkan çökme sebep olan satırdan çok uzakta görünür.
- **PECS:** okumak için üretici `extends`, yazmak için tüketici `super`.
- **`? extends` yazılamaz; `? super` `Object` olarak okunur.**
- **Jenerikler değişmez, diziler değişkendir.** Dizi seçimi hataydı ve `ArrayStoreException` onun faturası.

Ödev

[homework/README.md](#)

Jenerik bir kap kurun, üç tip silme kısıtına çarpın ve bilerek yığın kirlenmesine sebep olun. Referans çözüm [solutions/11-generics/](#) altında.